

Week 2

1. Introduction to Pointers

A pointer is a variable that stores the address of another variable.

Why pointers? They allow direct memory access, are used with arrays, functions, and dynamic memory, and make programs more flexible.

2. Basic Operations with Pointers

Declaration:

int *ptr; (Recommended)

int * ptr;

int* ptr;

Initialization: ptr = &x;

Dereferencing: *ptr gives value stored at that address.

Example 1: Simple pointer use

```
#include <iostream>
using namespace std;
int main() {
    int a = 5;
    int *p = &a;
    cout << "Value of a: " << a << endl;
    cout << "Address of a: " << p << endl;
    cout << "Value using pointer: " << *p << endl;
    return 0;
}
```

Output:

Value of a: 5

Address of a: 0x... (some memory address)

Value using pointer: 5

3. Pointer Arithmetic

You can move pointers using ++ or --. When a pointer is incremented, it points to the next memory location of that data type.

Example 2: Pointer arithmetic

```
#include <iostream>
using namespace std;
int main() {
    int arr[3] = {10, 20, 30};
    int *p = arr;
    cout << *p << endl;
    p++;
    cout << *p << endl;
    p++;
    cout << *p << endl;
    return 0;
}
```

Output:

```
10
20
30
```

4. Pointer to Pointer

A pointer can also store the address of another pointer.

Syntax: int **pp;

Example 3: Double pointer

```
#include <iostream>
using namespace std;
int main() {
    int x = 42;
    int *p = &x;
    int **pp = &p;
    cout << "Value of x: " << **pp;
    return 0;
}
```

Output:

```
Value of x: 42
```

5. Pointers and Functions

Passing pointers to functions allows modifying the original data.

Example 4: Swap using pointers

```
#include <iostream>
using namespace std;
void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}
int main() {
    int x = 10, y = 20;
    swap(&x, &y);
    cout << "x=" << x << " y=" << y;
    return 0;
}
```

Output:

x=20 y=10

6. Strings and Pointers

A string is basically an array of characters. Using char * we can traverse strings.

Example 5: Count vowels using char pointer

```
#include <iostream>
using namespace std;
int main() {
    char str[] = "Hello World";
    char *p = str;
    int count = 0;
    while(*p != 0) {
        char ch = tolower(*p);
        if(ch=='a' || ch=='e' || ch=='i' || ch=='o' || ch=='u')
            count++;
        p++;
    }
    cout << "Vowels: " << count;
    return 0;
}
```

Output:
Vowels: 3

Practice Tasks (15)

1. Declare an int pointer and assign it to an integer variable. Print value and address.
2. Repeat the above with a float and a char variable.
3. Create a program to add two numbers using pointers.
4. Swap two numbers using pointers.
5. Reverse an array using pointers.
6. Count vowels in a string using char pointers.
7. Copy one string into another using pointers.
8. Concatenate two strings using pointers.
9. Find the largest element of an array using pointers.
10. Calculate average of an array using pointers.
11. Implement bubble sort using pointers.
12. Implement binary search using pointers.
13. Sum all elements of a 2D matrix using pointers.
14. Transpose a 2D matrix using pointers.
15. Write a pointer-to-pointer program that prints the value of an integer.